

INFORME DE AVANCE
Proyecto Integrador: Videojuego
"Defensa Estelar"

Integrantes del grupo de trabajo:

Leyes, Nahuel
Romero,
Ignacio
Gutierrez,
Lionel

Mendoza, 17 de junio de 2026

Introducción

El presente informe tiene como finalidad documentar el estado de avance del proyecto integrador correspondiente a la materia Programación Avanzada, consistente en el desarrollo de un videojuego del género conocido como “tower defense”, titulado Defensa Estelar.

El proyecto se desarrolla sobre el motor Godot 4, empleando como lenguaje de programación GDScript. Una particularidad del desarrollo es que la totalidad de los elementos visuales del juego

—mapas, torres, enemigos y efectos— son generados mediante código en tiempo de ejecución, sin recurrir a recursos gráficos externos.

A continuación se exponen los objetivos perseguidos, la arquitectura de software empleada, las mecánicas de juego implementadas hasta el momento, el estado actual del desarrollo y la evidencia correspondiente al trabajo realizado.

Objetivos

Los objetivos planteados para el desarrollo del presente proyecto son los siguientes:

- Desarrollar un videojuego completo y jugable del género tower defense, aplicando los conceptos de programación orientada a objetos abordados en la cátedra.
- Diseñar una arquitectura de software modular, separando responsabilidades mediante el empleo de singletons (Autoloads).
- Aplicar herencia de clases para evitar la duplicación de código entre las distintas torres y los distintos enemigos del juego.
- Incorporar patrones de optimización, tales como el reciclado de objetos (Object Pooling), para mejorar el rendimiento durante la ejecución.
- Implementar sistemas de progresión y persistencia de datos entre partidas.

Arquitectura del proyecto

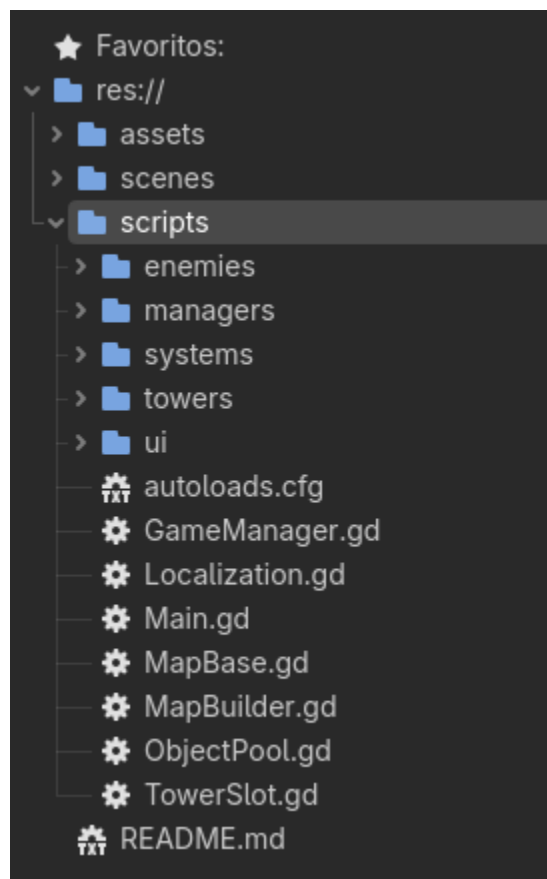
El proyecto se organiza siguiendo una separación clara entre las escenas del motor (archivos .tscn) y la lógica de programación asociada a cada una de ellas (archivos .gd).

Organización de carpetas

La estructura general del proyecto se detalla en la siguiente tabla:

Organización de carpetas del proyecto

Carpeta	Contenido
scenes/	Escenas raíz, de torres, de enemigos y de mapas.
scripts/towers/	Clase base de torre y las cuatro torres especializadas.
scripts/enemies/	Clase base de enemigo y los cinco tipos de enemigos.
scripts/managers/	Gestores de oleadas, de torres y de proyectiles.
scripts/systems/	Sistema de mejoras (árbol tecnológico).
scripts/ui/	Interfaz de usuario: menú, HUD, árbol tecnológico y pantalla de resultados.



Singletons (Autoloads)

Para la gestión de los sistemas transversales del juego se emplean cinco singletons, configurados como Autoload dentro del motor Godot, lo cual permite que sean accedidos desde cualquier escena sin necesidad de mantener referencias explícitas entre nodos. Su detalle se presenta en la Tabla 2.

Singletons (Autoloads) del proyecto

Autoload	Función que cumple
GameManager	Estado global del juego: recursos, vidas, oleadas, puntuación, dificultad, logros y guardado de datos.

Localization	Administración de los textos del juego en idioma español e inglés.
MapBuilder	Generación procedural de los mapas y sus respectivos caminos.
ObjectPool	Reciclado de proyectiles a fin de optimizar el rendimiento.
UpgradeSystem	Árbol tecnológico de mejoras permanentes entre partidas.

Diseño orientado a objetos

Tanto las torres de defensa como los enemigos del juego se modelan a partir de una clase base de la cual heredan las clases específicas, evitando de este modo la duplicación de código:

- La clase `Tower.gd` centraliza el comportamiento común a toda torre: adquisición de objetivo, cadencia de disparo, niveles de mejora y venta. De ella heredan `LaserTower`, `PlasmaTower`, `CryoTower` y `MissileTower`, cada una sobrescribiendo los parámetros y efectos que le son propios.

- La clase Enemy.gd centraliza la vida, el desplazamiento sobre el camino y la recepción de daño. De ella heredan AlienScout, AlienWarrior, AlienArmored, AlienFlyer y ColossalBoss, este último incorporando regeneración de vida y generación de enemigos secuaces.
- De modo análogo, la clase Projectile.gd constituye la base de los proyectiles, de la cual heredan CryoProjectile (efecto de ralentización) y MissileProjectile (daño en área).

Reciclado de objetos (Object Pooling)

Con el propósito de evitar la creación y destrucción constante de instancias de proyectiles durante el combate, lo cual resulta costoso en términos de rendimiento, se implementó un patrón de reciclado de objetos a través del Autoload ObjectPool.gd. De esta manera, los proyectiles son reutilizados en lugar de ser instanciados y eliminados en cada disparo.

Mecánicas de juego implementadas

Torres de defensa

El jugador dispone de cuatro tipos de torres, cuyas características se resumen en la Tabla 3.

Torres de defensa disponibles

Torre	Costo	Daño	Alcance	Cadencia	Particularidad
Láser	80	Bajo	Medio	Alta	Impacto instantáneo.
Plasma	150	Alto	Largo	Baja	Proyectil lento de alto impacto.
Cryo	100	Bajo	Corto	Media	Ralentiza a los enemigos un 50%.
Misil	130	Alto	Largo	Media	Provoca daño en área por explosión.

Cabe señalar que todas las torres cuentan con tres niveles de mejora y pueden ser vendidas, recuperándose en tal caso el sesenta por ciento de la inversión realizada.

Enemigos

Se incorporaron cinco tipos de enemigos alienígenas, cuyos atributos y comportamientos diferenciados se detallan en la siguiente tabla:

Tipos de enemigos implementados

Enemigo	Vida	Velocidad	Particularidad
Explorador	Baja	Alta	Sin características adicionales.
Guerrero	Media	Media	Sin características adicionales.
Blindado	Alta	Lenta	Posee armadura, lo que reduce en un 30% el daño recibido.
Volador	Media	Alta	Ignora el camino trazado y se desplaza en línea recta.
Jefe Colosal	2000	Muy lenta	Regenera su vida y genera enemigos secuaces.

Sistemas de progresión

Además de las torres y los enemigos descritos, el juego incorpora los siguientes sistemas:

- Veinte oleadas en el modo de juego normal, presentándose un enemigo de tipo jefe cada cinco oleadas.
- Un árbol tecnológico mediante el cual el jugador invierte puntos técnicos obtenidos durante la partida en mejoras permanentes, tales como incremento de daño global, de rango, de cadencia de disparo, de recursos obtenidos y de vidas base (véase Figura 3).
- Un modo de juego infinito, que se desbloquea al completarse las veinte oleadas del modo normal.
- Tres niveles de dificultad —Fácil, Normal y Difícil—, que modifican la vida y la velocidad de los enemigos, así como la cantidad de recursos otorgados al jugador.
- Cinco mapas con trazados de camino distintos entre sí.
- Un sistema de ocho logros desbloqueables, vinculados a hitos de progreso dentro del juego (véase Figura 4).
- Soporte completo de idioma español e inglés, con la posibilidad de alternar entre ambos desde el menú principal.
- Persistencia de las mejores puntuaciones obtenidas por mapa entre distintas sesiones de juego.

Estado actual del desarrollo

A la fecha de la presente entrega, el proyecto cuenta con una base jugable funcional. Se encuentran implementadas y operativas las cuatro torres descritas, los cinco tipos de enemigos, el sistema de oleadas, el árbol tecnológico, el sistema de logros y la localización en idioma español

e inglés, conforme puede verificarse en el video de demostración y en las capturas de pantalla adjuntas al presente informe.

El código fuente desarrollado hasta el momento asciende a aproximadamente tres mil líneas, distribuidas en veintisiete archivos de script, organizados conforme a la arquitectura descrita en el apartado tercero del presente informe.

Tareas pendientes hasta la mesa de examen

Resta llevar a cabo las siguientes tareas con vistas a la presentación final del proyecto:

- Incorporar y balancear los mapas numerados del dos al cinco, dado que a la fecha únicamente el primero de ellos —Desierto Alienígena— se encuentra completamente implementado en escena.
- Desarrollar las mejoras del árbol tecnológico que actualmente figuran como “próximamente” (escudo y disparo múltiple).
- Incorporar efectos de sonido y música ambiente, de los que el proyecto carece a la fecha.
- Ajustar el balance de dificultad entre los tres niveles disponibles.
- Incorporar elementos visuales y animaciones adicionales, manteniendo el criterio procedural empleado hasta el momento.
- Realizar pruebas de integración finales y corregir los errores menores detectados durante las pruebas de juego.

Evidencia del trabajo realizado

Repositorio de código fuente

El código fuente completo del proyecto se encuentra disponible públicamente en el siguiente repositorio:

Repositorio: <https://github.com/IgnaaRo/Defensa-Estelar-Programacion-Avanzada.git>

Video de demostración

Se acompaña asimismo un video de demostración del funcionamiento actual del juego:

Enlace al video: <https://youtu.be/leEhYnmA6jQ>

Capturas de pantalla

A continuación se presentan capturas de pantalla correspondientes al estado actual del desarrollo.



Figura 1. Menú principal del juego, con selección de mapa, dificultad y modo infinito.

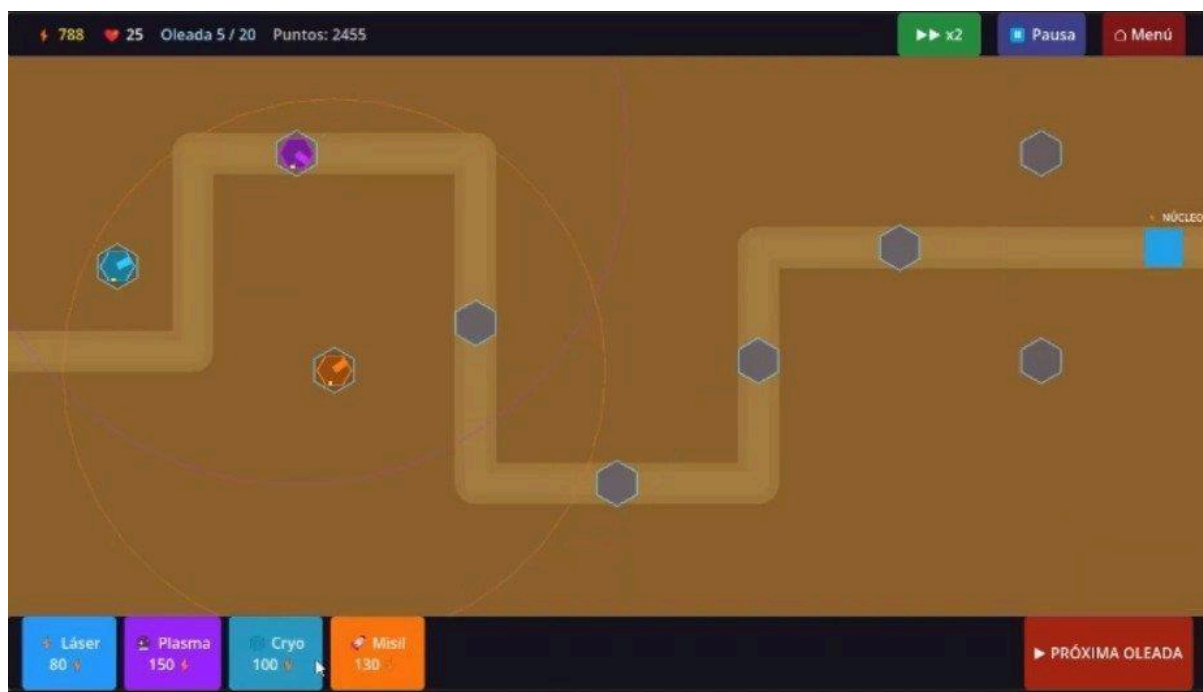


Figura 2. Partida en curso sobre el Mapa 1, en la quinta de veinte oleadas.



Figura 3. Laboratorio de Investigación, correspondiente al árbol tecnológico de mejoras.



Figura 4. Panel de logros desbloqueables del juego.

Conclusión

En virtud de lo expuesto, puede afirmarse que el desarrollo del proyecto Defensa Estelar se encuentra en un estado avanzado. El núcleo jugable correspondiente al género “tower defense” se

halla completamente funcional, contando con mecánicas de construcción y mejora de torres, una variedad de enemigos con comportamientos diferenciados, sistemas de progresión entre partidas y elementos de soporte tales como el sistema de logros, la selección de idioma y los niveles de dificultad.

El trabajo que resta llevar a cabo se concentra fundamentalmente en la incorporación de contenido adicional —mapas restantes y mejoras tecnológicas pendientes— y en el pulido general de la experiencia de juego, quedando el proyecto en condiciones de ser completado satisfactoriamente con vistas a su presentación en la mesa de examen correspondiente.